实验报告lab1 叶凯森

```
Epoch 1/10:   0%|                                                                                | 0/938 [00:00<?, ?it/s]
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:277: RuntimeWarning: divide by zero encountered in matmul
  out = x @ self.weight + self.bias
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:277: RuntimeWarning: overflow encountered in matmul
  out = x @ self.weight + self.bias
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:277: RuntimeWarning: invalid value encountered in matmul
  out = x @ self.weight + self.bias
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:286: RuntimeWarning: divide by zero encountered in matmul
  self.w_grad = self.x.T @ dy
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:286: RuntimeWarning: overflow encountered in matmul
  self.w_grad = self.x.T @ dy
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:286: RuntimeWarning: invalid value encountered in matmul
  self.w_grad = self.x.T @ dy
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:288: RuntimeWarning: divide by zero encountered in matmul
  dx = dy @ self.weight.T
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:288: RuntimeWarning: overflow encountered in matmul
  dx = dy @ self.weight.T
/Users/kaisenye/Desktop/2025summer_exp1/datacode/module.py:288: RuntimeWarning: invalid value encountered in matmul
```

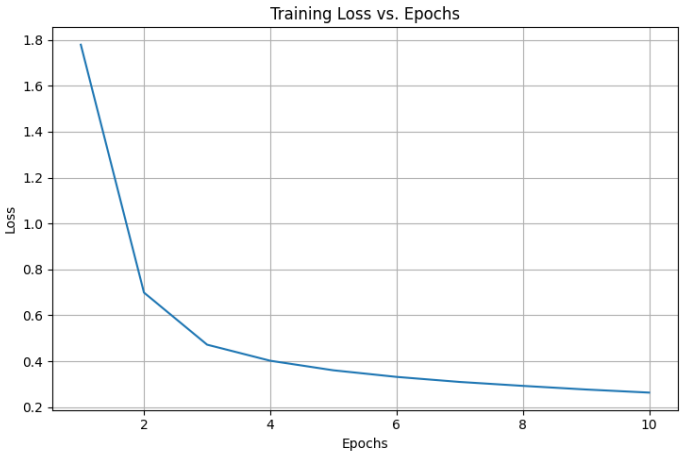
第一次的代码爆了很多runtimewarining，根源是**梯度爆炸 (Exploding Gradients)**。

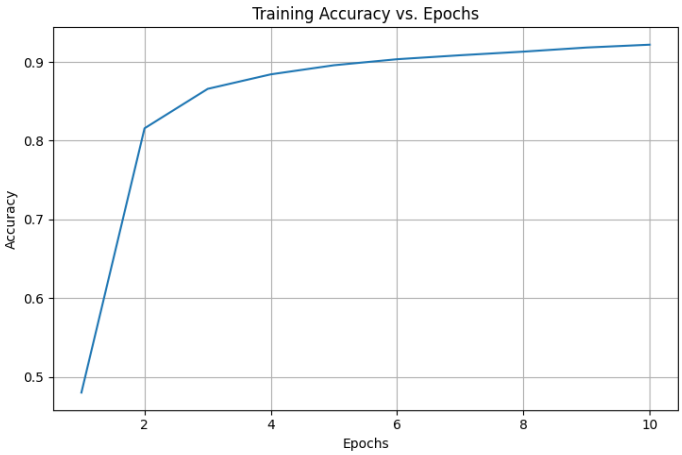
在反向传播过程中，梯度会逐层传递和累积。如果梯度值持续大于1，经过多层网络后，它会以指数级增长，变得异常巨大。这个巨大的梯度在更新权重时，会导致权重也变得异常巨大（即`overflow`），最终导致整个计算过程充满 `inf` 和 `NaN`

修改学习率为0.001

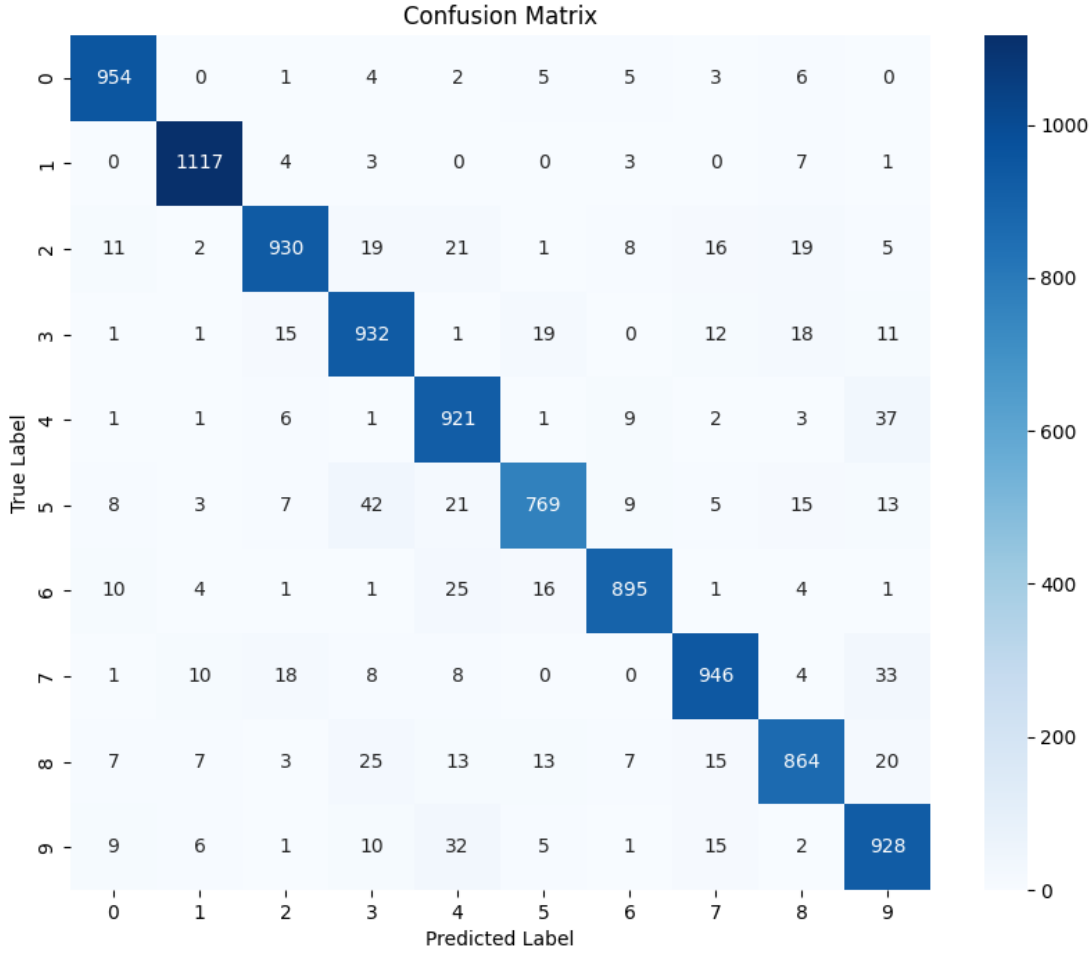
分类报告:

	precision	recall	f1-score	support
Digit 0	0.95	0.97	0.96	980
Digit 1	0.97	0.98	0.98	1135
Digit 2	0.94	0.90	0.92	1032
Digit 3	0.89	0.92	0.91	1010
Digit 4	0.88	0.94	0.91	982
Digit 5	0.93	0.86	0.89	892
Digit 6	0.96	0.93	0.94	958
Digit 7	0.93	0.92	0.93	1028
Digit 8	0.92	0.89	0.90	974
Digit 9	0.88	0.92	0.90	1009
accuracy			0.93	10000
macro avg	0.93	0.92	0.92	10000
weighted avg	0.93	0.93	0.93	10000

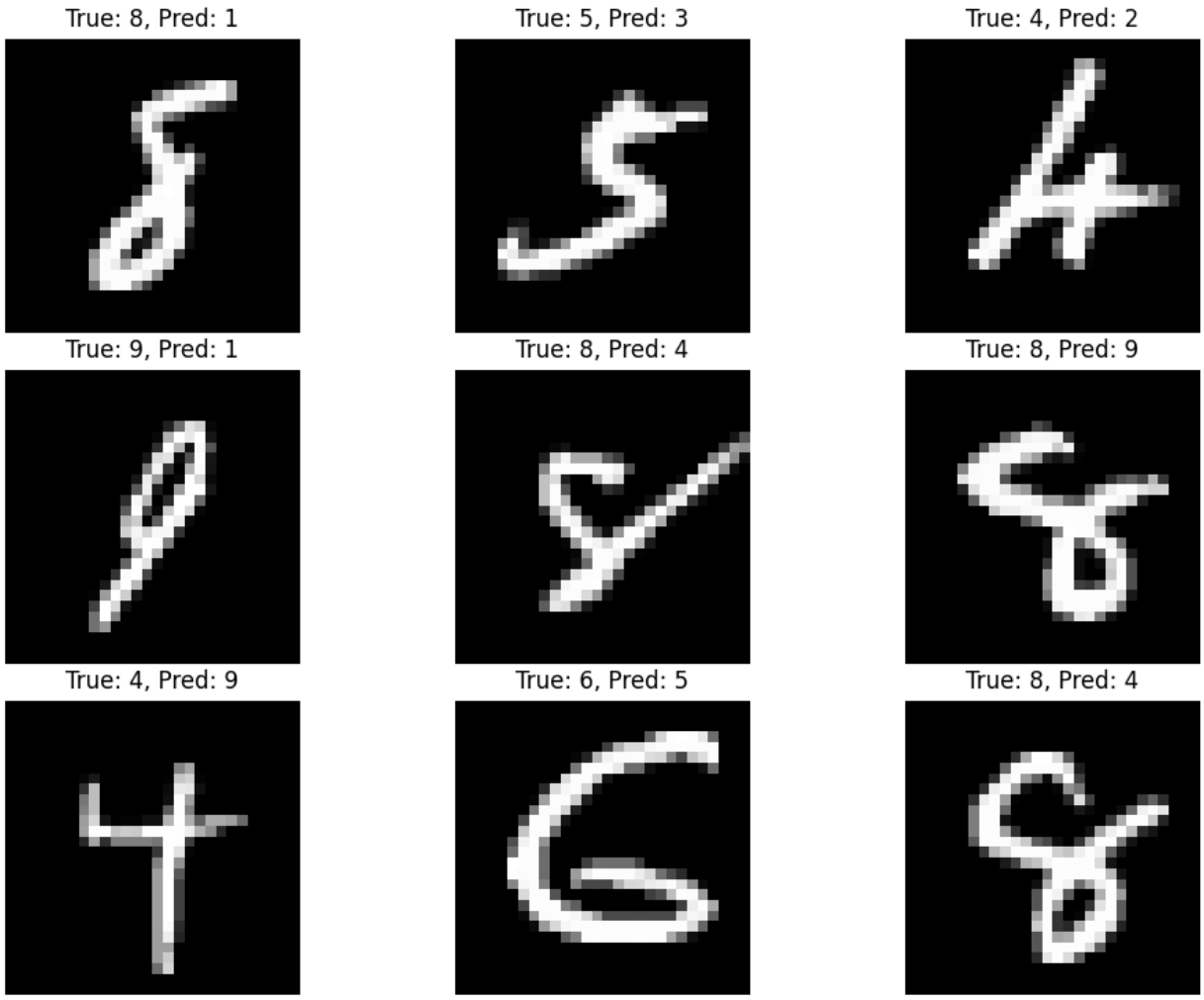




混淆矩阵



随机展示 9 个预测错误的样本:



改进代码：

```
def cosine_learning_rate(epoch, total_epochs, initial_lr):
    """
    计算给定epoch的余弦退火学习率
    """
    import math
    return initial_lr * 0.5 * (1 + math.cos(math.pi * epoch / total_epochs))
```

```
Epoch 6/10: 100%|██████████| 938/938 [00:21<00:00, 44.40it/s, acc=0.9063, loss=0.7737, lr=0.00050]
Epoch 6 完成: 平均损失 = 0.3272, 训练准确率 = 0.9063

Epoch 7/10: 100%|██████████| 938/938 [00:21<00:00, 44.07it/s, acc=0.9094, loss=0.2405, lr=0.00035]
Epoch 7 完成: 平均损失 = 0.3152, 训练准确率 = 0.9094

Epoch 8/10: 100%|██████████| 938/938 [00:22<00:00, 40.97it/s, acc=0.9116, loss=0.5184, lr=0.00021]
Epoch 8 完成: 平均损失 = 0.3079, 训练准确率 = 0.9116

Epoch 9/10: 100%|██████████| 938/938 [00:22<00:00, 41.49it/s, acc=0.9124, loss=0.1941, lr=0.00010]
Epoch 9 完成: 平均损失 = 0.3037, 训练准确率 = 0.9124

Epoch 10/10: 100%|██████████| 938/938 [00:22<00:00, 42.60it/s, acc=0.9130, loss=0.1593, lr=0.00002]
Epoch 10 完成: 平均损失 = 0.3020, 训练准确率 = 0.9130

📈 正在生成训练过程可视化图表...

🔬 开始在测试集上进行最终评估...
✅ 测试完成！最终测试准确率: 0.9206
```

虽然看不到明显的准确率提升，但这可能是因为模型的极限已经差不多这样了，原来的小学习率经过足够训练的模型已经足够拟合。但是这种动态学习率在大规模训练时有利于前期的快速训练和后期的微小调整。

采用miniresnet

```
class MiniResNet:
    def __init__(self):
        # 初始的卷積層
        self.conv1 = Conv2d(1, 16, kernel_size=3, stride=1, padding=1)
        self.relu = ReLU()

        # 堆疊殘差塊
        self.res_block1 = ResidualBlock(16, 16, stride=1)
        self.res_block2 = ResidualBlock(16, 32, stride=2) # stride=2 會將尺寸減半
        self.res_block3 = ResidualBlock(32, 64, stride=2) # 再次將尺寸減半

        # 使用全局平均池化層和最終的全連接層
        self.avg_pool = AvgPool2d(kernel_size=7)
        self.fc = Linear(in_features=64, out_features=10)

    def forward(self, x): # x shape: (N, 1, 28, 28)
        x = self.relu.forward(self.conv1.forward(x)) # -> (N, 16, 28, 28)

        x = self.res_block1.forward(x)      # -> (N, 16, 28, 28)
        x = self.res_block2.forward(x)      # -> (N, 32, 14, 14)
        x = self.res_block3.forward(x)      # -> (N, 64, 7, 7)
        x = self.avg_pool.forward(x)        # -> (N, 64, 1, 1)
        x = self.fc.forward(x)              # -> (N, 10)
        return x

    def backward(self, dy, lr):
        dy = self.fc.backward(dy, lr)
        dy = self.avg_pool.backward(dy)
        dy = self.res_block3.backward(dy, lr)
        dy = self.res_block2.backward(dy, lr)
        dy = self.res_block1.backward(dy, lr)
        dy = self.relu.backward(dy)
        self.conv1.backward(dy, lr)
```

并通过numba进行加速

```
# 在 module.py 文件顶部
import numpy as np
from numba import njit # 1. 导入 njit

# ...

# 2. 在函數定義上方加上 @njit
@njit
def im2col(image, kernel_h, kernel_w, stride):
```

```
    # ... (函數內部程式碼完全不變) ...

# 3. 在函數定義上方加上 @njit
@njit
def col2im(col, input_shape, kernel_h, kernel_w, stride):
    # ... (函數內部程式碼完全不變) ...

# ... (後續的類定義不變) ...
```

```
🚀 开始训练...
Epoch 1/15: 100%|██████████| 938/938 [03:08<00:00,  4.98it/s, acc=0.1513, loss=2.1904]
Epoch 1 完成: 平均损失 = 2.2588, 训练准确率 = 0.1513

Epoch 2/15: 100%|██████████| 938/938 [03:08<00:00,  4.97it/s, acc=0.4006, loss=1.5890]
Epoch 2 完成: 平均损失 = 2.0174, 训练准确率 = 0.4006

Epoch 3/15: 100%|██████████| 938/938 [03:07<00:00,  4.99it/s, acc=0.6196, loss=0.7477]
Epoch 3 完成: 平均损失 = 1.3276, 训练准确率 = 0.6196

Epoch 4/15: 100%|██████████| 938/938 [02:56<00:00,  5.33it/s, acc=0.4237, loss=nan]
```

但是训练效果不佳，可能是细节没做到位。